ФЕДЕРАЛЬНОЕ АГЕНТСТВО ПО ОБРАЗОВАНИЮ

Государственное образовательное учреждение высшего
профессионального образования

«Белгородский государственный технологический университет
им. В.Г. Шухова»

Институт ИТУС
Кафедра информационных технологий

Курсовая работа
по дисциплине «Информатика»
на тему:

«Разработка клавиатурного тренажера»

Выполнил: студент группы ИТЗ-221
Дмитриев Дмитрий Анатольевич
Проверил: старший преподаватель
Коломыцева Елена Павловна

Белгород 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 3
1. ОБЩАЯ ИНФОРМАЦИЯ И КРАТКАЯ ОЦЕНКА 3
1.1. Актуальность разработки клавиатурного тренажера 3
1.2. Обзор существующих клавиатурных тренажеров 3
1.3. Цель и задачи работы 4
1.4. Краткое описание вопросов, решенных в ходе выполнения работы 4
ОСНОВНАЯ ЧАСТЬ 5
2. ПОСТАНОВКА ЗАДАЧИ И ОПИСАНИЕ ИСХОДНЫХ ДАННЫХ 5
2.1. Описание задачи 5
2.2. Исходные данные 5
3. СВЕДЕНИЯ, НЕОБХОДИМЫЕ ДЛЯ РЕШЕНИЯ ЗАДАЧИ 7
3.1. Основные принципы слепой печати 7
3.2. Принципы работы клавиатурных тренажеров 8
3.3. Принципы реализации подсчета скорости и анализа ошибок 8
4. ОПИСАНИЕ ПОДХОДОВ К РЕШЕНИЮ ЗАДАЧИ 9
4.1. Описание возможных подходов 9
4.2. Обоснование выбранного подхода 9
4.3. Описание алгоритма решения задачи 10
5. ОПИСАНИЕ СТРУКТУРЫ ДАННЫХ И АЛГОРИТМОВ 11
5.1. Структура данных 11
5.2. Алгоритм решения задачи 11
5.3. Спецификация алгоритмов 15
ЗАКЛЮЧЕНИЕ 17
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ И ЛИТЕРАТУРЫ 18
ПРИЛОЖЕНИЕ А 19
ПРИЛОЖЕНИЕ Б 27

ВВЕДЕНИЕ

1. ОБЩАЯ ИНФОРМАЦИЯ И КРАТКАЯ ОЦЕНКА

1.1. Актуальность разработки клавиатурного тренажера

В современном мире, где технологии стали неотъемлемой частью повседневной жизни, навыки работы с компьютером играют ключевую роль в эффективной профессиональной и личной деятельности. Одним из важнейших элементов компьютерной грамотности является умение быстро и точно печатать на клавиатуре. Это важное качество, которое влияет на производительность и удобство работы, особенно в тех профессиях, где работа с текстами и данными является основной частью деятельности.

Современные тренажеры зачастую предлагают ограниченные функции или не соответствуют требованиям пользователей с различным уровнем подготовки. На данный момент существует множество онлайн-ресурсов и программ, однако они не всегда могут предложить оптимальный способ тренировки для конкретного пользователя, не учитывая его индивидуальные особенности.

В современных условиях важно не только обеспечить доступ к обучению, но и создать инструменты, которые смогут эффективно отслеживать процесс обучения, индивидуализировать задания и предоставлять пользователю возможность самому контролировать прогресс в обучении [1].

Актуальность работы обусловлена необходимостью улучшения навыков печати, которые являются неотъемлемой частью профессиональной деятельности в современном мире. Задача создания клавиатурного тренажера актуальна, поскольку многие существующие решения либо не предлагают достаточной гибкости, либо имеют ограничения по функционалу. Работа актуальна, так как решает важную задачу повышения производительности и эффективности работы с компьютером.

1.2. Обзор существующих клавиатурных тренажеров

Сегодня существует множество тренажеров для обучения слепой печати, как в виде десктопных программ, так и онлайн-сервисов. Наиболее известными программами являются Stamina, TypingMaster и онлайн-сервисы, например TypingClub. Эти программы предлагают различные подходы к обучению: от стандартных уроков с набором текста до более сложных тренажеров с анализом ошибок и статистикой.

Однако несмотря на свою популярность, эти тренажеры часто имеют ограничения, такие как платные подписки и ограничения по количеству использований. Таким образом, есть необходимость в создании более гибкой и адаптированной программы, которая будет учитывать индивидуальные особенности каждого пользователя. Существующие решения не всегда удовлетворяют

потребности пользователей, что подтверждает необходимость создания более гибкого и адаптивного тренажера.

1.3. Цель и задачи работы

Целью данной работы является разработка клавиатурного тренажера, который позволит пользователю эффективно развивать навыки быстрой печати. Такой тренажер должен обеспечивать комплексный подход к обучению, включая оценку скорости печати, подсчет ошибок и предоставление статистики.

Для достижения этой цели необходимо решить следующие задачи:

- разработать интерфейс программы с возможностью выбора сложности;
- реализовать загрузку словарей и случайный выбор заданий для ввода пользователем;
- реализовать алгоритмы для подсчета скорости печати и ошибок.

В ходе работы решил задачи по созданию интерфейса, реализации подсчета скорости и анализа ошибок, а также настройке параметров тренировки по выбранной сложности.

1.4. Краткое описание вопросов, решенных в ходе выполнения работы

В ходе выполнения работы разработал алгоритмы для вывода букв, слогов и слов, анализа ошибок и подсчета скорости ввода. Создал оконный интерфейс с выбором сложности. Для подготовки заданий использовал русские словари букв, слогов и слов. Их элементы считываются из ресурсов программы и случайным образом подбираются для отображения на экране.

ОСНОВНАЯ ЧАСТЬ

2. ПОСТАНОВКА ЗАДАЧИ И ОПИСАНИЕ ИСХОДНЫХ ДАННЫХ

2.1. Описание задачи

Основная цель разработки программы - создание клавиатурного тренажера, который помогает пользователю улучшить скорость и точность печати. Программа должна случайным образом выводить на экран буквы, слоги и слова, которые необходимо набрать на клавиатуре. Для ввода каждого очередного символа отводится определенное время в зависимости от уровня сложности.

Программа должна оценивать производительность пользователя по количеству правильно введенных символов, а также учитывать ошибки. За каждый правильно введенный символ начисляется одно очко, а за ошибки происходит списание баллов. Оценка производится по количеству правильных нажатий, что позволяет пользователю следить за своим прогрессом и тренироваться.

Задача программы заключается в следующем:

- случайным образом генерировать наборы символов, слогов и слов, которые пользователю необходимо ввести;
- оценивать скорость ввода и точность на основе правильных и неправильных символов;
- отображать результат тренировки с учетом времени, потраченного на ввод, и количества ошибок;
- обеспечить три уровня сложности, в которых меняются такие параметры, как время на ввод символа, количество секунд на тест и минимальное количество баллов для прохождения теста.

Ключевая цель программы - помочь пользователю улучшить навыки печати через тренировки с временными ограничениями.

2.2. Исходные данные

Для генерации текста и оценки работы программы используются следующие исходные данные:

- словари символов, слогов и слов. Программа использует русские словари для случайного выбора заданий. Словарь букв содержит все 33 буквы русского алфавита, остальные словари содержат русские слоги и слова. Один и тот же набор словарей используется на всех уровнях сложности;
- алгоритм генерации символов, слогов и слов. Программа использует функцию генерации, которая случайным образом выбирает элементы из словарей и формирует из них тренировочные задания;
- технические данные. Для корректной работы программы также важны параметры, такие как настройки времени, отведенного на ввод символа (в

зависимости от уровня сложности), количество очков, требуемых для прохождения теста, и лимит времени на выполнение теста.

Каждый уровень сложности будет изменять время, отведенное на ввод одного символа:

- легкий уровень: 3 секунды на символ, 40 секунд на тест, цель 40 очков, штраф 1 очко за ошибку;
- средний уровень: 2 секунды на символ, 30 секунд на тест, цель 60 очков, штраф 2 очка за ошибку;
- сложный уровень: 1 секунда на символ, 25 секунд на тест, цель 80 очков, штраф 3 очка за ошибку.

Выбор исходных данных и их организация имеют важное значение для адекватной работы тренажера, обеспечивая его гибкость и функциональность. Таким образом, программа генерирует случайные наборы символов, слогов и слов, учитывая параметры сложности, и оценивает пользователя по количеству правильных вводов, что позволяет тренировать навыки быстрой печати и точности.

3. СВЕДЕНИЯ, НЕОБХОДИМЫЕ ДЛЯ РЕШЕНИЯ ЗАДАЧИ

3.1. Основные принципы слепой печати

Слепая печать является ключевым навыком для эффективного взаимодействия с компьютером, особенно в профессиях, где требуется быстрая и точная работа с текстом. Этот процесс заключается в наборе текста без использования визуального контакта с клавишами, что позволяет пользователю сфокусироваться исключительно на экране и минимизировать время на поиски клавиш. Как отмечают исследователи в области компьютерной грамотности, освоение слепой печати значительно увеличивает скорость набора текста и снижает количество ошибок [2].

Основным принципом слепой печати является использование всех десяти пальцев, что повышает общую продуктивность и позволяет достигать высоких результатов в быстром наборе. Существуют различные методики обучения, такие как программированные тренажеры, которые предполагают шаг за шагом изучение расположения клавиш с постепенным увеличением сложности.

К принципам относятся:

- сидеть необходимо прямо, не нагибаясь и не наклоняя голову. Угол между позвоночником и бедром и между бедром и голенью должен составлять 90 градусов. Взгляд должен быть направлен вперёд, в центр экрана монитора. Расстояние от глаз до экрана монитора - от 40 до 70 сантиметров. Правильная поза очень важна для здоровья и способствует быстрому обучению печати слепым методом;
- за каждым пальцем при обучении слепому методу печати закреплены свои клавиши, которые должны нажиматься именно им. Исходное положение левой руки: мизинец на клавише «Ф», безымянный на «Ы», средний на «В», указательный на «А». Исходное положение правой руки: мизинец на «Ж», безымянный на «Д», средний на «Л», указательный на «О». Большие пальцы обеих рук располагаются на клавише пробела;
- «ФЫВА» и «ОЛДЖ» - это так называемые домашние клавиши. Чтобы можно было быстро сориентироваться, как расположить руки в стартовой позиции, на клавишах «А» и «О» делают небольшие выступы;
- заниматься изучением десятипальцевого метода набора вслепую на компьютере желательно два-три раза в день. Для начала можно печатать свою фамилию, расположив пальцы на домашней строке и не глядя на клавиатуру. Перемещать необходимо только пальцы. Взгляд должен быть направлен не на клавиатуру, а на экран [3].

Освоение слепой печати является ключевым навыком для повышения производительности работы с компьютером, что подтверждает актуальность создания тренажеров для ее улучшения.

3.2. Принципы работы клавиатурных тренажеров

Клавиатурные тренажеры создаются для того, чтобы пользователи могли развивать свои навыки печати через целенаправленную практику и обратную связь. Важным аспектом является эффективная оценка результатов, которая позволяет пользователю увидеть свой прогресс, что значительно увеличивает мотивацию к дальнейшему обучению.

Основными характеристиками тренажеров являются:

- оценка скорости печати - это важнейшая метрика, которая измеряется количеством символов, набранных за единицу времени;
- оценка точности - точность набора измеряется количеством правильно введенных символов по сравнению с общим числом символов. Ошибки снижают общий результат;
- сложность упражнений - для более эффективного обучения важно постепенно увеличивать сложность упражнений, начиная с простых слов и заканчивая сложными текстами. Сложность может регулироваться временем на ввод каждого символа, минимальным количеством правильных вводов или количеством ошибок.

Современные тренажеры используют различные подходы для обучения, включая подсчет скорости печати и анализ ошибок. Также важным аспектом является персонализация тренировки, что позволяет адаптировать задания в зависимости от уровня пользователя. Эти принципы являются основой для эффективного обучения и тренировки.

3.3. Принципы реализации подсчета скорости и анализа ошибок

Один из ключевых аспектов работы клавиатурного тренажера заключается в оценке скорости ввода и анализе ошибок. Обычно скорость выражают в символах в минуту. В данной программе расчетный показатель зависит от набранных очков, поэтому на него влияют как правильные нажатия, так и штрафы за ошибки.

Важным фактором является также учет времени на ввод каждого символа. Программа должна точно отслеживать, сколько времени ушло на ввод каждого символа и сколько времени осталось для всего теста. В зависимости от этого можно корректировать оценку и подсчитывать баллы.

Анализ ошибок помогает пользователю замечать неверные нажатия. В программе за каждый ошибочный символ увеличивается счетчик ошибок и списывается установленное количество очков. Размер штрафа зависит от выбранного уровня сложности.

Также важно, чтобы программа отслеживала, сколько символов пользователь успел ввести до окончания отведенного времени. Это требует точной синхронизации между временем и введенными символами.

Эти принципы и алгоритмы лежат в основе любого эффективного клавиатурного тренажера и являются основой для реализации функционала, необходимого для выполнения поставленной задачи.

Точная оценка ошибок и времени необходима для мотивации пользователей и корректного анализа их прогресса.

4. ОПИСАНИЕ ПОДХОДОВ К РЕШЕНИЮ ЗАДАЧИ

4.1. Описание возможных подходов

Для клавиатурного тренажера выбрал оконное приложение под Windows. В одном окне разместил выбор сложности, параметры тренировки, задание, поле ввода и результаты. Пользователь видит нужную букву и получает результаты после каждого нажатия.

Оконный интерфейс на C++ можно создать средствами WinAPI, MFC или Qt Widgets. При использовании WinAPI расположение элементов и обработку сообщений приходится подробнее описывать в коде. MFC предоставляет классы для работы с окнами Windows и редактор диалогов. Qt Widgets позволяет собрать форму из готовых элементов и связать их с логикой программы [4].

Для этой работы выбрал Qt Widgets. Форму подготовил в визуальном редакторе, а обработку действий пользователя написал на C++. Такой подход позволил отдельно изменять расположение элементов и логику тренировки. Работа приложения выполняется на платформе Windows, проверку на других операционных системах не выполнял.

4.2. Обоснование выбранного подхода

Программу написал на C++. Для интерфейса использовал Qt 6: основное окно, надписи QLabel, список QComboBox, поле QLineEdit, кнопку QPushButton и область QGroupBox. Этих элементов достаточно для выбора сложности, показа задания, приема нажатий и отображения результата [4].

Описание формы сохранил в файле mainwindow.ui. В редакторе формы задал расположение элементов, подписи, шрифты и размеры окна. Минимальную и максимальную ширину и высоту установил одинаковыми, поэтому пользователь не может изменить размер окна. Область задания выделил белым фоном.

Для реакции на кнопку и смену сложности использовал обработчики сигналов. Сигнал позволяет сообщить о действии в интерфейсе, а связанный с ним слот выполняет нужную функцию [5]. Нажатие кнопки запускает или останавливает тренировку. Изменение выбранной сложности обновляет параметры следующего теста.

Для сборки проекта использовал CMake. В настройках перечислены исходные файлы, форма, файл ресурсов и необходимые компоненты Qt. Русские словари добавил в resources.qrc, чтобы программа читала их из своих ресурсов, а не искала рядом с запущенным файлом [6].

Логику тренировки разместил в классе основного окна. Для небольшого учебного приложения этого достаточно: отдельные функции загружают словари, выбирают задание, проверяют время и обновляют надписи.

4.3. Описание алгоритма решения задачи

При создании окна программа загружает три русских словаря: буквы, слоги и слова. Каждую непустую строку добавляет в соответствующий список. Если хотя бы один словарь пуст или не загрузился, выводится сообщение об ошибке, а кнопка начала тренировки отключается. После успешной загрузки списки объединяются.

До запуска пользователь выбирает сложность. Программа показывает общую длительность теста, время на символ, целевое количество очков и размер штрафа. После нажатия кнопки счетчики обнуляются, запускаются отсчеты времени и выбирается первое задание. Сменить сложность во время тренировки нельзя.

При нажатии печатной клавиши программа сначала проверяет время, затем сравнивает введенный символ с текущей буквой задания. Правильное нажатие увеличивает счет и переводит позицию вперед. Неверное нажатие увеличивает число ошибок и уменьшает очки, но не изменяет уже введенный текст. Контроль нажатий реализовал через фильтр событий поля ввода [7].

После последней буквы выбирается следующее задание. Проверка продолжается до истечения общего времени, превышения времени на символ или ручной остановки. В конце программа показывает причину завершения, количество ошибок, очки и расчетную скорость. Для исходного кода предусмотрел приложение А, для блок-схем основных алгоритмов - приложение Б.

5. ОПИСАНИЕ СТРУКТУРЫ ДАННЫХ И АЛГОРИТМОВ

5.1. Структура данных

Для хранения словарей использовал QStringList, то есть списки строк Qt. Словарь букв содержит 33 строки, словарь слогов содержит 108 строк, а словарь слов содержит 60 строк. Все элементы записаны на русском языке. При загрузке удаляются пробелы по краям строк и пропускаются пустые строки. Для чтения указал кодировку UTF-8. После загрузки объединил списки в общий набор из 201 элемента. Случайный индекс выбирается из всего набора. Отдельного выбора вида задания нет: пользователю могут последовательно попадаться буквы, слоги и слова. Уровень сложности влияет на ограничения и штраф, но не меняет состав словарей. Текущее слово храню в строке, а позицию ожидаемого символа - в целочисленной переменной. В начале задания позиция равна нулю. После правильного нажатия она увеличивается на единицу. Когда позиция достигает длины строки, программа выбирает новый элемент из общего списка.

Для настроек и текущего состояния использовал переменные. В них храню длительность теста, время на символ, целевые очки, штраф, оставшееся время, текущий счет и количество ошибок. Логическая переменная показывает, идет ли тренировка. Для времени использовал два объекта QElapsedTimer: один отсчитывает время всего теста, второй - время на очередной символ. Периодическую проверку вызывает QTimer с интервалом 50 мс. QElapsedTimer измеряет прошедшее время, а QTimer вызывает обработчик через заданные интервалы [8].

5.2. Алгоритм решения задачи

В верхней части окна разместил название тренажера и список сложности. Под списком показал параметры выбранного режима. В центральной белой области находятся задание, поле ввода и поясняющая надпись. Внизу расположены оставшееся время, счет и кнопка управления. Начальное окно тренажера показано на рисунке 5.1.

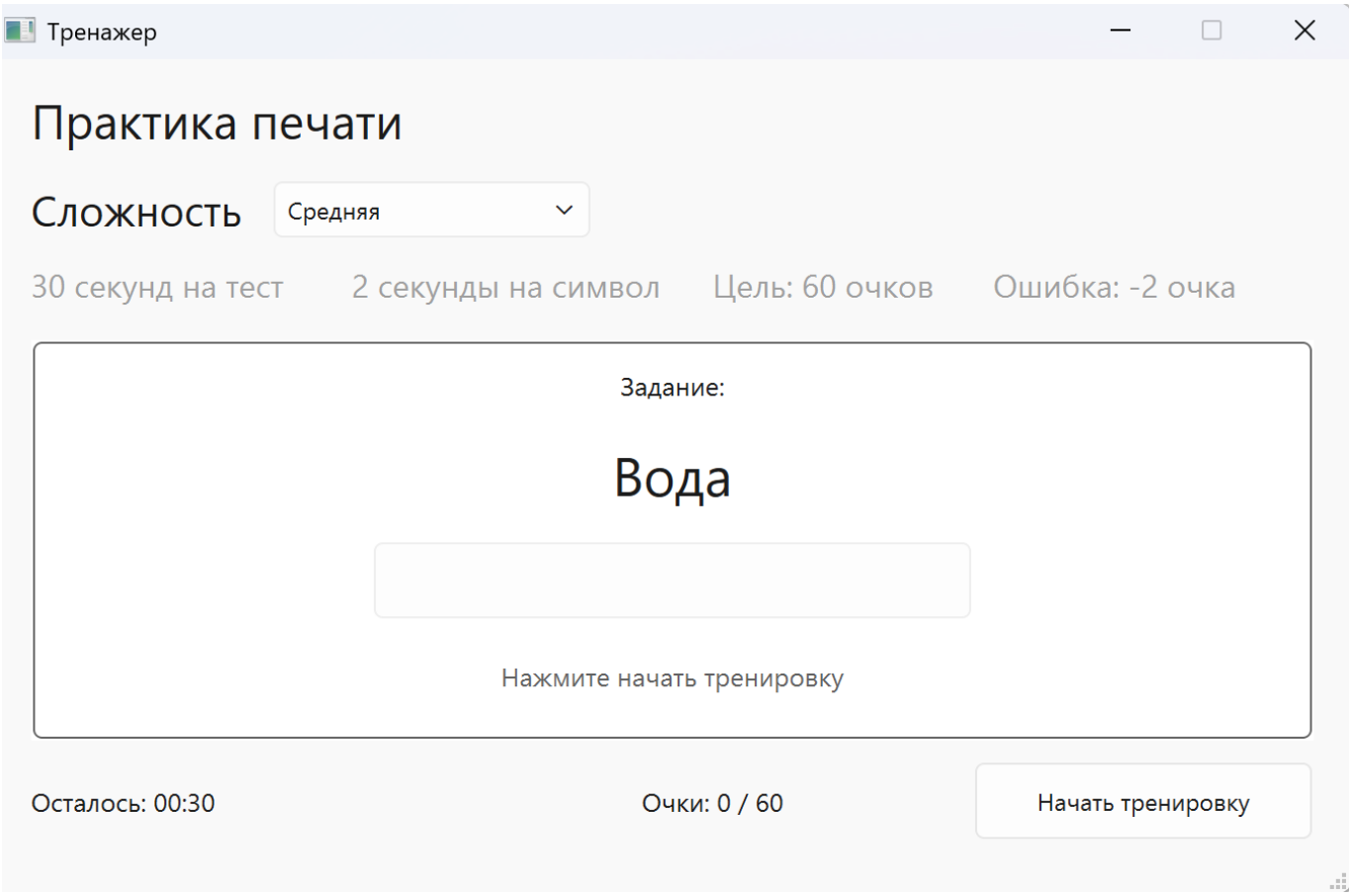


Рисунок 5.1 Начальное окно и выбор сложности

До начала тренировки поле ввода отключено. После нажатия «Начать тренировку» программа сбрасывает очки и ошибки, запускает оба отсчета времени и выбирает случайное задание. Поле становится доступным для обработки нажатий и получает фокус, а надпись кнопки меняется на «Завершить». Обычное редактирование текста в поле отключил. Содержимое поля программа обновляет самостоятельно только после правильного нажатия. Фильтр событий получает печатный символ и сравнивает его с ожидаемой буквой. Для подсветки задания формирую текст с цветовым оформлением. Уже введенные буквы выделяются зеленым цветом. Текущая буква отображается синим цветом на светлом фоне и подчеркивается. Остальные буквы остаются темными. После правильного нажатия подсветка переходит к следующей позиции. Подсветка символов во время тренировки показана на рисунке 5.2.

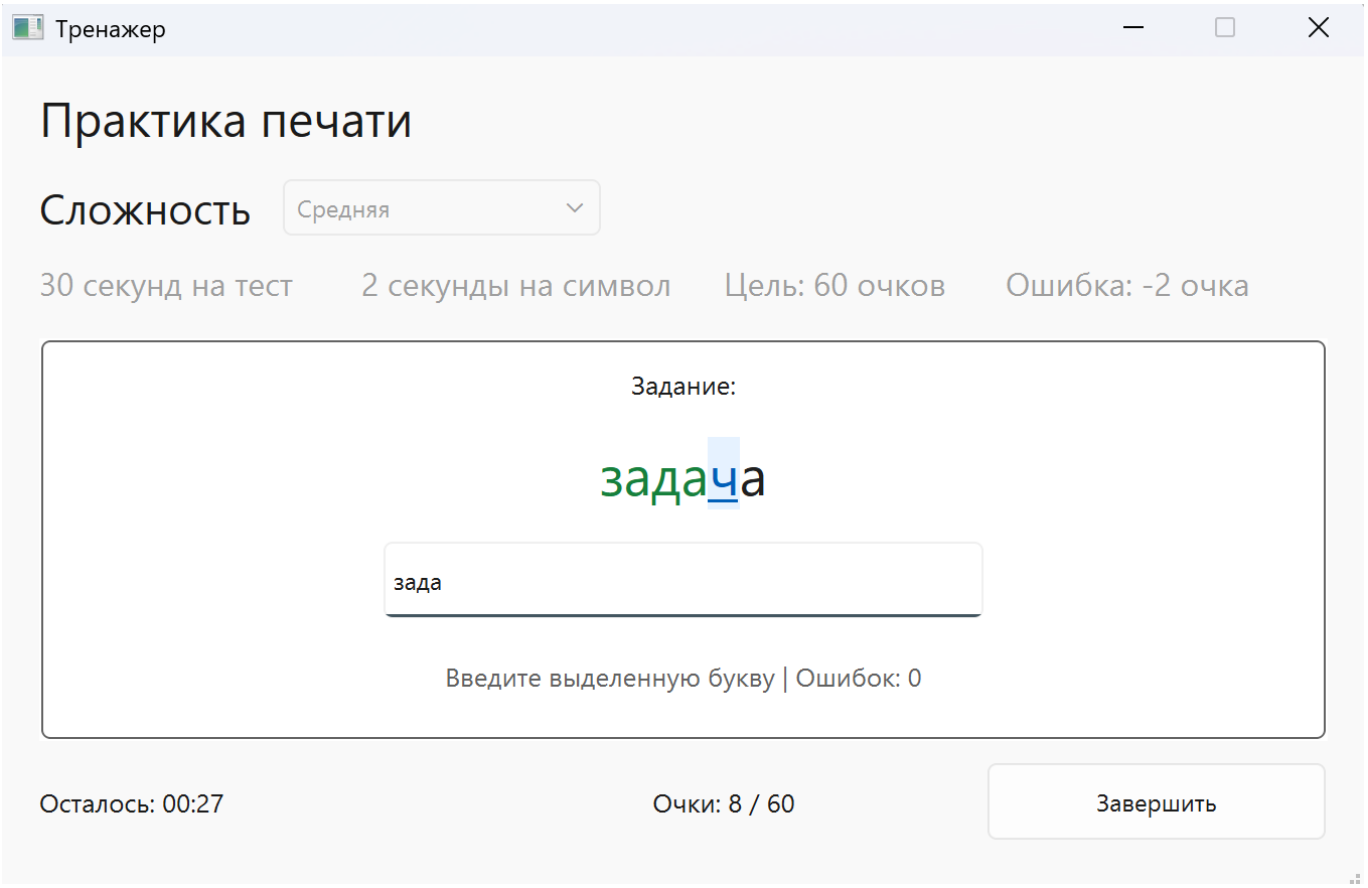


Рисунок 5.2 Подсветка символов во время тренировки

Если введен правильный символ, счет увеличивается на одно очко, а отсчет времени на символ начинается заново. Если слово или слог введен полностью, программа очищает поле и показывает новое задание. Ранее введенные задания в окне не сохраняются. При ошибке введенный символ не попадает в поле, позиция остается прежней, счетчик ошибок увеличивается, а из счета вычитается штраф выбранной сложности. В поясняющей надписи красным цветом показывается нужная буква. Время на символ при ошибке не сбрасывается. Пример сообщения при неверном вводе показан на рисунке 5.3.

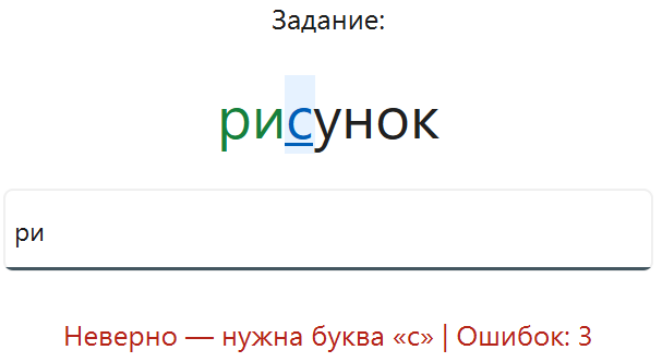


Рисунок 5.3 Сообщение при неверном вводе

Общее время и время на символ проверяются перед обработкой печатного нажатия и при срабатывании таймера. Если одно из ограничений достигнуто, дальнейший ввод прекращается. Сам интервал QTimer не является измерением прошедшего времени: фактическую длительность программа получает из QElapsedTimer.

Во время тренировки показываю оставшиеся секунды и текущие очки относительно цели. Отрицательный счет допускается, если штрафов оказалось больше, чем правильных нажатий. Достижение целевого счета само по себе не останавливает тренировку. Условие прохождения проверяется при автоматическом завершении. Если при автоматическом завершении счет не меньше цели, выводится зеленая надпись «Тест пройден». В противном случае показывается красная надпись «Тест не пройден». Причина завершения выводится отдельно: истекло общее время или время на букву. Результаты успешного и неуспешного завершения теста показаны на рисунках 5.4 и 5.5.

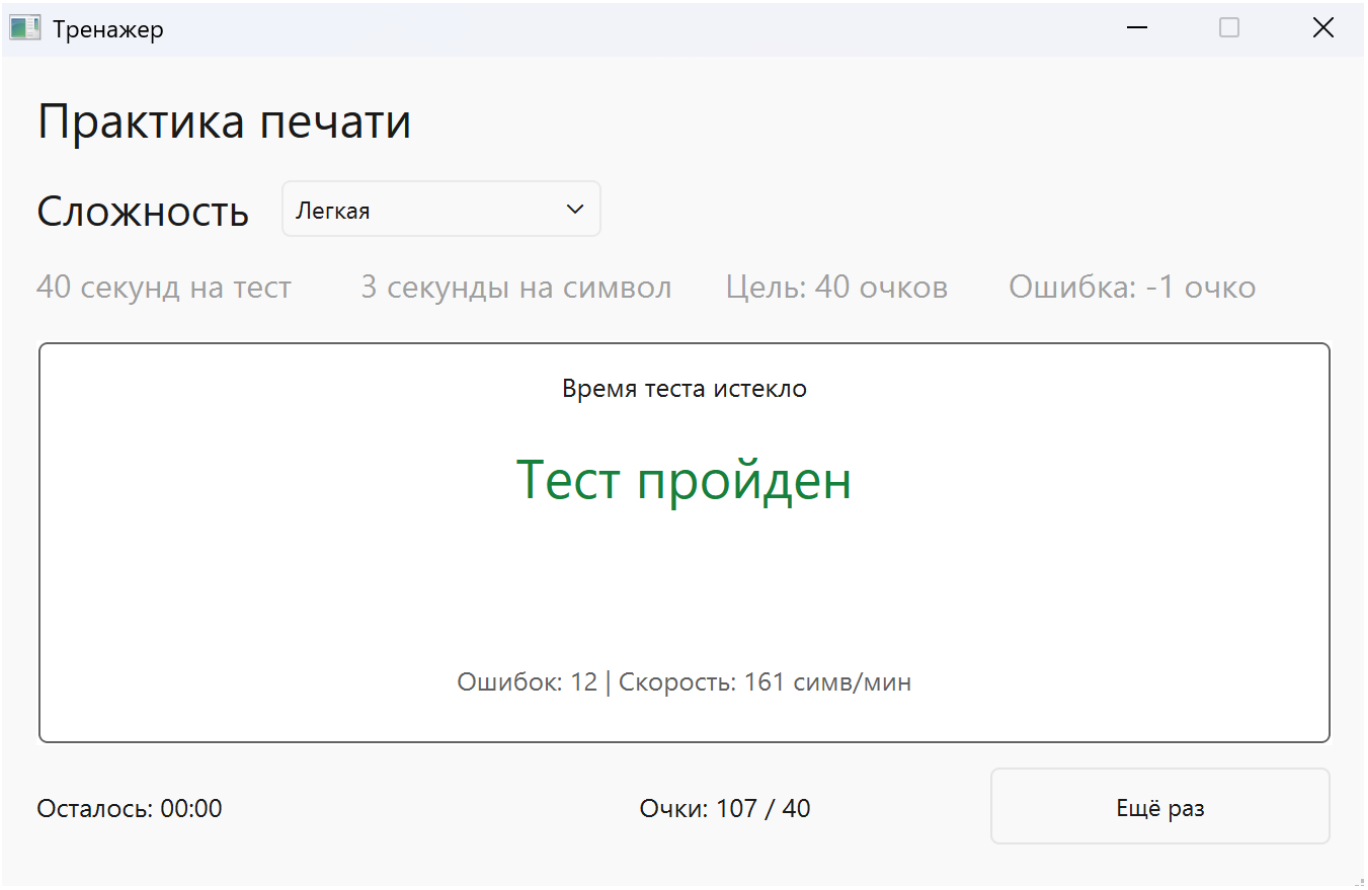


Рисунок 5.4 Успешное завершение теста

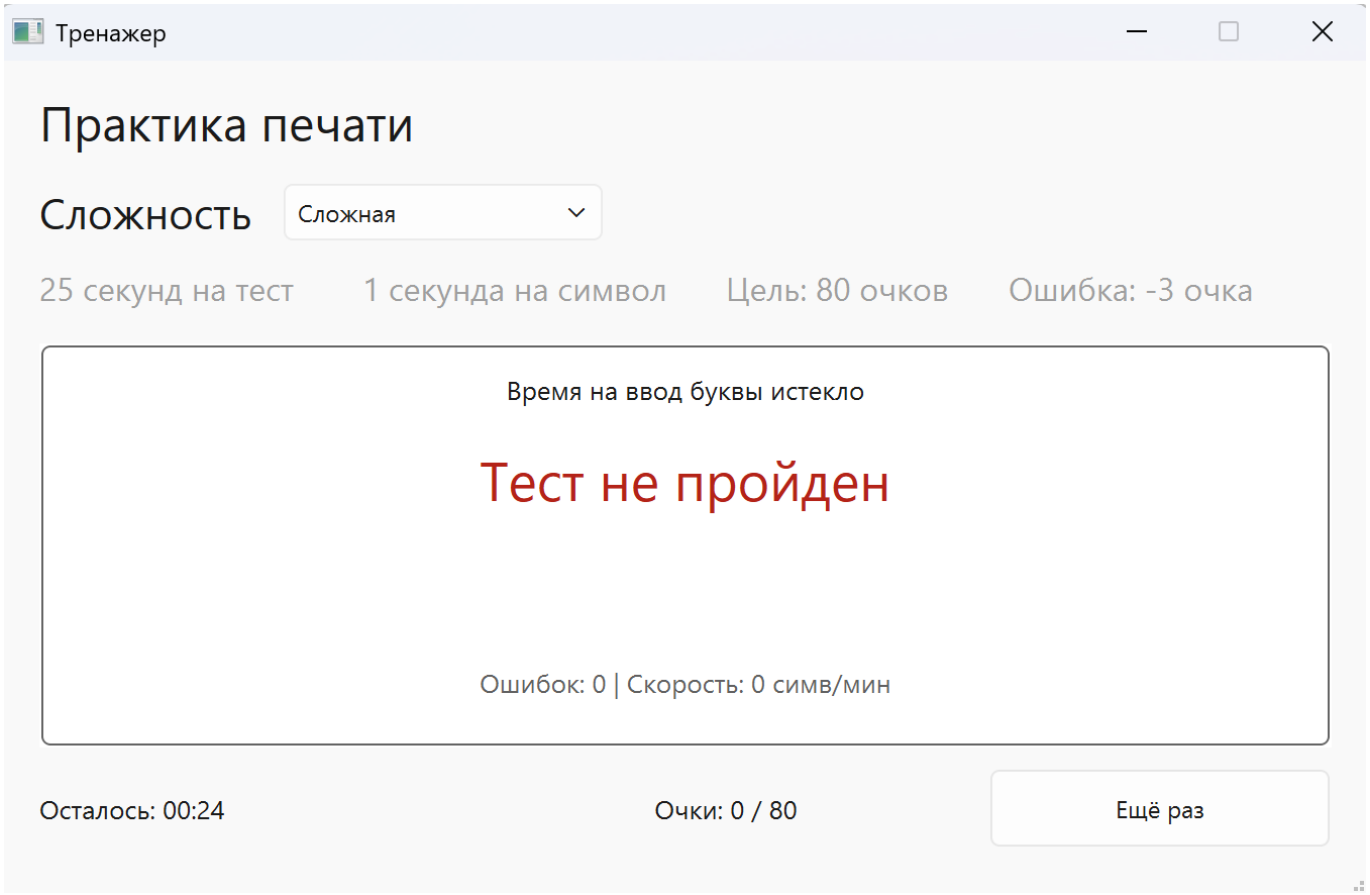


Рисунок 5.5 Неуспешное завершение теста

При ручном завершении программа показывает «Тест остановлен», даже если счет уже достиг цели. После окончания поле ввода скрывается, выбор сложности снова становится доступным, а на кнопке появляется «Ещё раз». Повторное нажатие начинает новую тренировку с обнулением предыдущих счетчиков.

Расчетную скорость получаю из итогового счета: умножаю число очков на 60, делю на прошедшее время в секундах и округляю результат до целого. Время ограничиваю общей длительностью выбранного теста. При неположительном счете или нулевой длительности скорость равна нулю. Так как счет учитывает штрафы, этот показатель отличается от числа правильно введенных символов в минуту.

5.3. Спецификация алгоритмов

Функция loadDictionary принимает путь к ресурсу и возвращает непустые строки файла UTF-8. Если открыть файл не удалось, результатом становится пустой список. Все три списка проверяю при создании окна.

Обработчик on_difficultyComboBox_currentIndexChanged задает параметры сложности, обновляет надписи и сбрасывает счетчики. Во время тренировки настройки не изменяет.

Обработчик on_startButton_clicked запускает тест из состояния ожидания. Во время тренировки он проверяет время и выполняет ручную остановку, если автоматическое завершение еще не наступило.

Функция showNextTask выбирает индекс через QRandomGenerator, записывает задание, обнуляет позицию и очищает поле. Функция updateTaskHighlight формирует цветное отображение букв с учетом текущей позиции.

В eventFilter сравниваю печатный символ с ожидаемым и изменяю счетчики. Сочетания с Ctrl, Alt или системной клавишей и непечатные клавиши передаю стандартной обработке. Произвольное редактирование поля остается отключенным.

Функция checkTime проверяет состояние теста и оба ограничения времени. При превышении вызывает finishTest и возвращает признак остановки. Функция updateTimer вызывает эту проверку и обновляет статистику.

Функция updateStatistics обновляет оставшиеся целые секунды и очки. Для времени не допускаю отрицательного значения. При расчете итоговой скорости использую прошедшие миллисекунды, переведенные в секунды.

Функция finishTest останавливает таймер, рассчитывает скорость и выбирает статус с учетом ручного завершения. Она скрывает поле ввода и возвращает выбор сложности. Повторный вызов после завершения ничего не меняет.

ЗАКЛЮЧЕНИЕ

В ходе курсовой работы разработал оконный клавиатурный тренажер на C++ с использованием Qt Widgets. Приложение предназначил для самостоятельной тренировки ввода русских букв, слогов и слов под Windows. Для достижения поставленной цели реализовал выбор сложности, подготовку заданий, проверку вводимых символов, учет ошибок и отображение статистики.

Для решения задачи разработки интерфейса подготовил форму, на которой разместил параметры теста, задание, поле ввода, оставшееся время и текущий счет. Предусмотрел три уровня сложности с разными ограничениями времени, целевыми очками и размером штрафа. Пользователь может выбирать подходящие условия тренировки в зависимости от своей подготовки. Запуск, ручное завершение и повторную тренировку организовал с помощью одной кнопки.

Для получения тренировочных заданий подготовил русские словари и подключил их к ресурсам программы. Реализовал загрузку непустых строк в кодировке UTF-8 и случайный выбор элементов из общего набора. Это позволило использовать в одной тренировке отдельные буквы, слоги и слова.

Реализовал последовательную проверку ввода и цветовую подсветку задания. Правильное нажатие увеличивает счет и переводит позицию к следующему символу. Неверная буква не принимается, увеличивает число ошибок и вызывает списание очков. Подсветка показывает уже введенную часть задания и букву, которую необходимо ввести следующей.

Для оценки результата реализовал подсчет очков, ошибок и расчетного показателя скорости с учетом штрафов. Добавил общий отсчет времени и ограничение на ввод каждого символа. При автоматическом завершении результат определяется сравнением счета с целью, а при ручной остановке выводится отдельное сообщение. После окончания пользователь может выбрать другую сложность и начать новую тренировку.

В результате выполнил задачи по созданию интерфейса, загрузке и случайному выбору заданий, подсчету ошибок и расчету статистики. По исходникам проверил соответствие описания реализованной логике. В ходе работы закрепил навыки программирования на C++, создания форм Qt, обработки событий клавиатуры и использования таймеров. Разработанный тренажер позволяет выполнять упражнения с временными ограничениями и получать текущую и итоговую оценку результата.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ И ЛИТЕРАТУРЫ

1. Розенберг, А. М. Компьютерная грамотность: учебное пособие / А. М. Розенберг. - 2-е изд., перераб. и доп. - Москва, 2012. - 352 с. - (Библиотека информационных технологий). (дата обращения: 21.02.2025).
2. Филиппов, Ю. В. Основы компьютерной грамотности: Учебное пособие / Ю. В. Филиппов, В. В. Груздев. - М.: Высшая школа, 2018. - 256 с. (дата обращения: 21.02.2025).
3. Метод слепой печати. Foxford / URL: https://externat.foxford.ru/polezno-znat/metod-slepoy-pechati (дата обращения: 25.02.2025).
4. Qt Widgets / Qt документация / URL: https://doc.qt.io/qt-6.5/qtwidgets-index.html (дата обращения: 27.02.2025).
5. Signals & Slots / Qt документация / URL: https://doc.qt.io/qt-6.5/signalsandslots.html (дата обращения: 27.02.2025).
6. The Qt Resource System / Qt документация / URL: https://doc.qt.io/qt-6.5/resources.html (дата обращения: 27.02.2025).
7. QObject Class. eventFilter / Qt документация / URL: https://doc.qt.io/qt-6.5/qobject.html#eventFilter (дата обращения: 28.10.2025).
8. QTimer Class / Qt Documentation / URL: https://doc.qt.io/qt-6.5/qtimer.html (дата обращения: 28.10.2025).
9. QElapsedTimer Class / Qt документация / URL: https://doc.qt.io/qt-6.5/qelapsedtimer.html (дата обращения: 28.10.2025).

ПРИЛОЖЕНИЕ А

Исходный текст программы

```
#include "mainwindow.h"
#include "ui_mainwindow.h"

#include <QFile>
#include <QEvent>
#include <QKeyEvent>
#include <QRandomGenerator>
#include <QStringConverter>
#include <QTextStream>
#include <QtMath>

MainWindow::MainWindow(QWidget *parent)
    : QMainWindow(parent)
    , ui(new Ui::MainWindow)
{
    ui->setupUi(this);

    testTimer = new QTimer(this);
    testTimer->setInterval(50);
    testTimer->setTimerType(Qt::PreciseTimer);
    connect(testTimer, &QTimer::timeout, this, &MainWindow::updateTimer);

    ui->typingLineEdit->setReadOnly(true);
    ui->typingLineEdit->setEnabled(false);
    ui->typingLineEdit->installEventFilter(this);

    ui->feedbackLabel->setEnabled(true);
    ui->taskTextLabel->setTextFormat(Qt::RichText);

    on_difficultyComboBox_currentIndexChanged(ui->difficultyComboBox-
>currentIndex());

    QStringList letters = loadDictionary(":/library/letters.ru.dic");
    QStringList syllables = loadDictionary(":/library/syllables.ru.dic");
    QStringList words = loadDictionary(":/library/words.ru.dic");

    if (letters.isEmpty() || syllables.isEmpty() || words.isEmpty()) {
        ui->feedbackLabel->setText(QStringLiteral("Не удалось загрузить словари или
один из них пуст"));
        ui->startButton->setEnabled(false);
        return;
    }
```

```
    trainingData = letters + syllables + words;
}

MainWindow::~MainWindow()
{
    testTimer->stop();
    delete ui;
}

void MainWindow::on_startButton_clicked()
{
    if (testRunning) {
        if (checkTime()) {
            finishTest(QStringLiteral("Остановлено пользователем"), true);
        }
        return;
    }

    if (trainingData.isEmpty()) {
        ui->feedbackLabel->setText(QStringLiteral("Словари пусты"));
        return;
    }

    score = 0;
    errorCount = 0;
    timeLeft = testDuration;
    testRunning = true;

    testClock.start();
    symbolClock.start();

    ui->difficultyComboBox->setEnabled(false);
    ui->startButton->setText("Завершить");
    ui->taskCaptionLabel->setText("Задание:");

    ui->typingLineEdit->show();
    ui->typingLineEdit->setEnabled(true);

    ui->feedbackLabel->setText("Введите выделенную букву | Ошибок: 0");

    showNextTask();
    updateStatistics();

    testTimer->start();

    ui->typingLineEdit->setFocus();
}
```

```
bool MainWindow::eventFilter(QObject *object, QEvent *event)
{
    if (object != ui->typingLineEdit || event->type() != QEvent::KeyPress) {
        return QMainWindow::eventFilter(object, event);
    }

    if (!testRunning) {
        return true;
    }

    QKeyEvent *keyEvent = static_cast<QKeyEvent *>(event);
    if (keyEvent->modifiers() & (Qt::ControlModifier | Qt::AltModifier | Qt::MetaModifier))
{
        return QMainWindow::eventFilter(object, event);
    }

    QString text = keyEvent->text();
    if (text.isEmpty() || !text.at(0).isPrint()) {
        return QMainWindow::eventFilter(object, event);
    }

    if (!checkTime()) {
        return true;
    }

    if (text.size() == 1 && text.at(0) == currentTask.at(taskPosition)) {
        score++;
        taskPosition++;
        symbolClock.restart();

        if (taskPosition == currentTask.size()) {
            showNextTask();
        } else {
            ui->typingLineEdit->setText(currentTask.left(taskPosition));
            ui->typingLineEdit->setCursorPosition(taskPosition);
            updateTaskHighlight();
        }

        ui->feedbackLabel->setStyleSheet(QStringLiteral("color: #616161;"));
        ui->feedbackLabel->setText(
            QStringLiteral("Введите выделенную букву | Ошибок: %1").arg(errorCount));
    } else {
        ++errorCount;
        score -= errorPenalty;
        ui->feedbackLabel->setStyleSheet(QStringLiteral("color: #b42318;"));
        ui->feedbackLabel->setText(
```

```
            QStringLiteral("Неверно — нужна буква «%1» | Ошибок: %2")
                .arg(currentTask.mid(taskPosition, 1)).arg(errorCount));
    }

    updateStatistics();
    return true;
}

void MainWindow::showNextTask()
{
    int index = QRandomGenerator::global()->bounded(static_cast<int>(trainingData.size()));
    currentTask = trainingData[index];
    taskPosition = 0;
    ui->typingLineEdit->clear();
    updateTaskHighlight();
}

void MainWindow::updateTaskHighlight()
{
    QString text;

    for (int i = 0; i < currentTask.size(); ++i) {
        QString letter = currentTask.mid(i, 1).toHtmlEscaped();

        if (i < taskPosition) {
            text += QStringLiteral("<span style=\"color: #16803c;\">%1</span>").arg(letter);
        } else if (i == taskPosition) {
            text += QStringLiteral("<span style=\"color: #005fb8; background-color: #e6f2ff; text-decoration: underline;\">%1</span>").arg(letter);
        } else {
            text += QStringLiteral("<span style=\"color: #202020;\">%1</span>").arg(letter);
        }
    }

    ui->taskTextLabel->setText(text);
}

void MainWindow::updateStatistics()
{
    if (testRunning) {
        timeLeft = testDuration - static_cast<int>(testClock.elapsed() / 1000);
        if (timeLeft < 0) {
            timeLeft = 0;
        }
    }
```

```
    ui->remainingTimeLabel->setText(
        QStringLiteral("Осталось: 00:%1").arg(timeLeft, 2, 10, QChar('0')));
    ui->scoreLabel->setText(QStringLiteral("Очки: %1 / %2").arg(score).arg(targetScore));
}

bool MainWindow::checkTime()
{
    if (!testRunning) {
        return false;
    }

    if (testClock.elapsed() >= testDuration * 1000) {
        finishTest(QStringLiteral("Время теста истекло"));
        return false;
    }

    if (symbolClock.elapsed() >= symbolTimeLimit * 1000) {
        finishTest(QStringLiteral("Время на ввод буквы истекло"));
        return false;
    }

    return true;
}

void MainWindow::updateTimer()
{
    if (checkTime()) {
        updateStatistics();
    }
}

void MainWindow::finishTest(const QString &reason, bool manuallyStopped)
{
    if (!testRunning) {
        return;
    }

    testTimer->stop();
    updateStatistics();
    testRunning = false;

    double elapsedSeconds = testClock.elapsed() / 1000.0;
    if (elapsedSeconds > testDuration) {
        elapsedSeconds = testDuration;
    }

    int speed = 0;
```

```
    if (elapsedSeconds > 0 && score > 0) {
        speed = qRound(score * 60.0 / elapsedSeconds);
    }

    ui->taskCaptionLabel->setText(reason);
    if (manuallyStopped) {
        ui->taskTextLabel->setText(QStringLiteral("Тест остановлен"));
    } else if (score >= targetScore) {
        ui->taskTextLabel->setText(QStringLiteral("<span style=\"color: #16803c;\">Тест пройден</span>"));
    } else {
        ui->taskTextLabel->setText(QStringLiteral("<span style=\"color: #b42318;\">Тест не пройден</span>"));
    }

    ui->typingLineEdit->setEnabled(false);
    ui->typingLineEdit->hide();
    ui->difficultyComboBox->setEnabled(true);
    ui->startButton->setText(QStringLiteral("Ещё раз"));
    ui->feedbackLabel->setStyleSheet(QStringLiteral("color: #616161;"));
    ui->feedbackLabel->setText(
        QStringLiteral("Ошибок: %1 | Скорость: %2 симв/мин").arg(errorCount).arg(speed));
    ui->startButton->setFocus();
}

QStringList MainWindow::loadDictionary(const QString &fileName)
{
    QStringList dictionary;
    QFile file(fileName);

    if (!file.open(QIODevice::ReadOnly)) {
        return dictionary;
    }

    QTextStream stream(&file);
    stream.setEncoding(QStringConverter::Utf8);

    while (!stream.atEnd()) {
        QString text = stream.readLine().trimmed();
        if (!text.isEmpty()) {
            dictionary.append(text);
        }
    }

    return dictionary;
}
```

```
void MainWindow::on_difficultyComboBox_currentIndexChanged(int index)
{
    if (testRunning) {
        return;
    }

    switch (index) {
    case 0:
        testDuration = 40;
        symbolTimeLimit = 3;
        targetScore = 40;
        errorPenalty = 1;
        break;
    case 1:
        testDuration = 30;
        symbolTimeLimit = 2;
        targetScore = 60;
        errorPenalty = 2;
        break;
    case 2:
        testDuration = 25;
        symbolTimeLimit = 1;
        targetScore = 80;
        errorPenalty = 3;
        break;
    default:
        return;
    }

    const QString secondsWord = symbolTimeLimit == 1
        ? QStringLiteral("секунда")
        : QStringLiteral("секунды");
    const QString pointsWord = errorPenalty == 1
        ? QStringLiteral("очко")
        : QStringLiteral("очка");

    ui->testDurationLabel->setText(
        QStringLiteral("%1 секунд на тест").arg(testDuration));
    ui->symbolTimeLimitLabel->setText(
        QStringLiteral("%1 %2 на символ").arg(symbolTimeLimit).arg(secondsWord));
    ui->targetScoreLabel->setText(
        QStringLiteral("Цель: %1 очков").arg(targetScore));
    ui->errorPenaltyLabel->setText(
        QStringLiteral("Ошибка: -%1 %2").arg(errorPenalty).arg(pointsWord));
    score = 0;
    errorCount = 0;
```

```
    taskPosition = 0;
    timeLeft = testDuration;
    ui->taskCaptionLabel->setText(QStringLiteral("Задание:"));
    ui->taskTextLabel->setText(QStringLiteral("Вода"));
    ui->typingLineEdit->clear();
    ui->typingLineEdit->show();
    ui->typingLineEdit->setEnabled(false);
    ui->feedbackLabel->setStyleSheet(QStringLiteral("color: #616161;"));
    ui->feedbackLabel->setText(QStringLiteral("Нажмите начать тренировку"));
    ui->startButton->setText(QStringLiteral("Начать тренировку"));
    updateStatistics();
}
```

ПРИЛОЖЕНИЕ Б

Блок-схемы алгоритмов программы

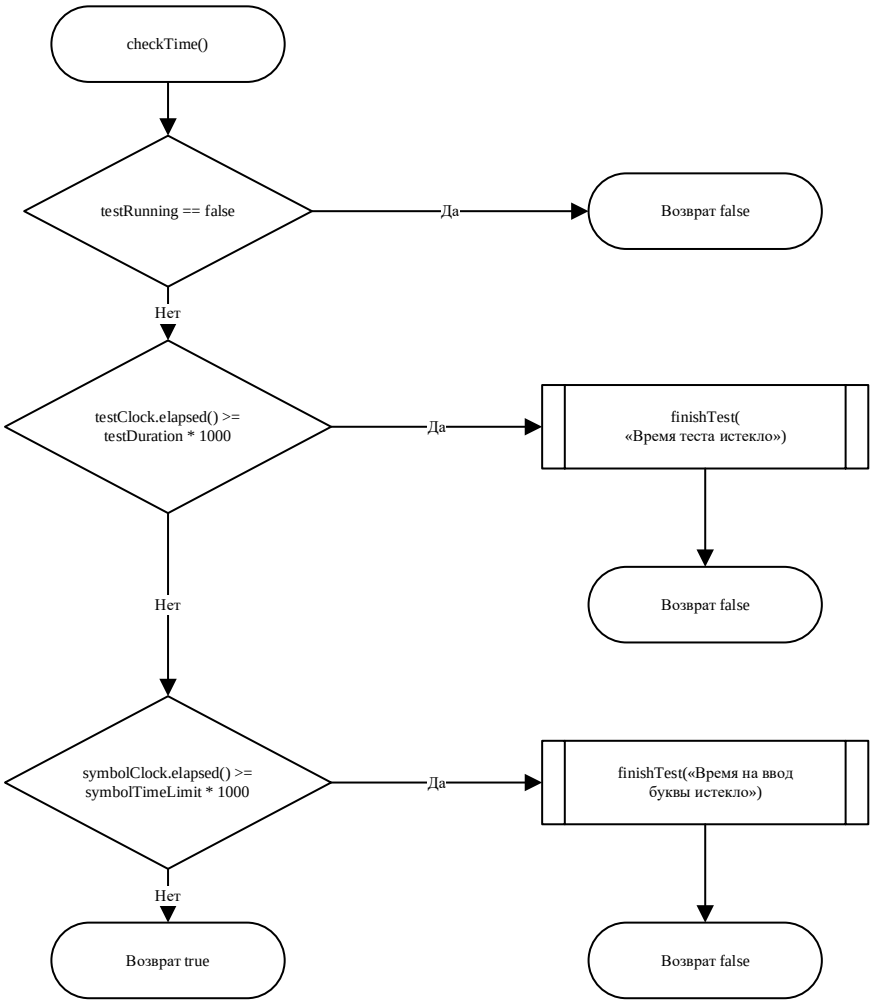


Рисунок Б.1 Контроль общего и символьного времени

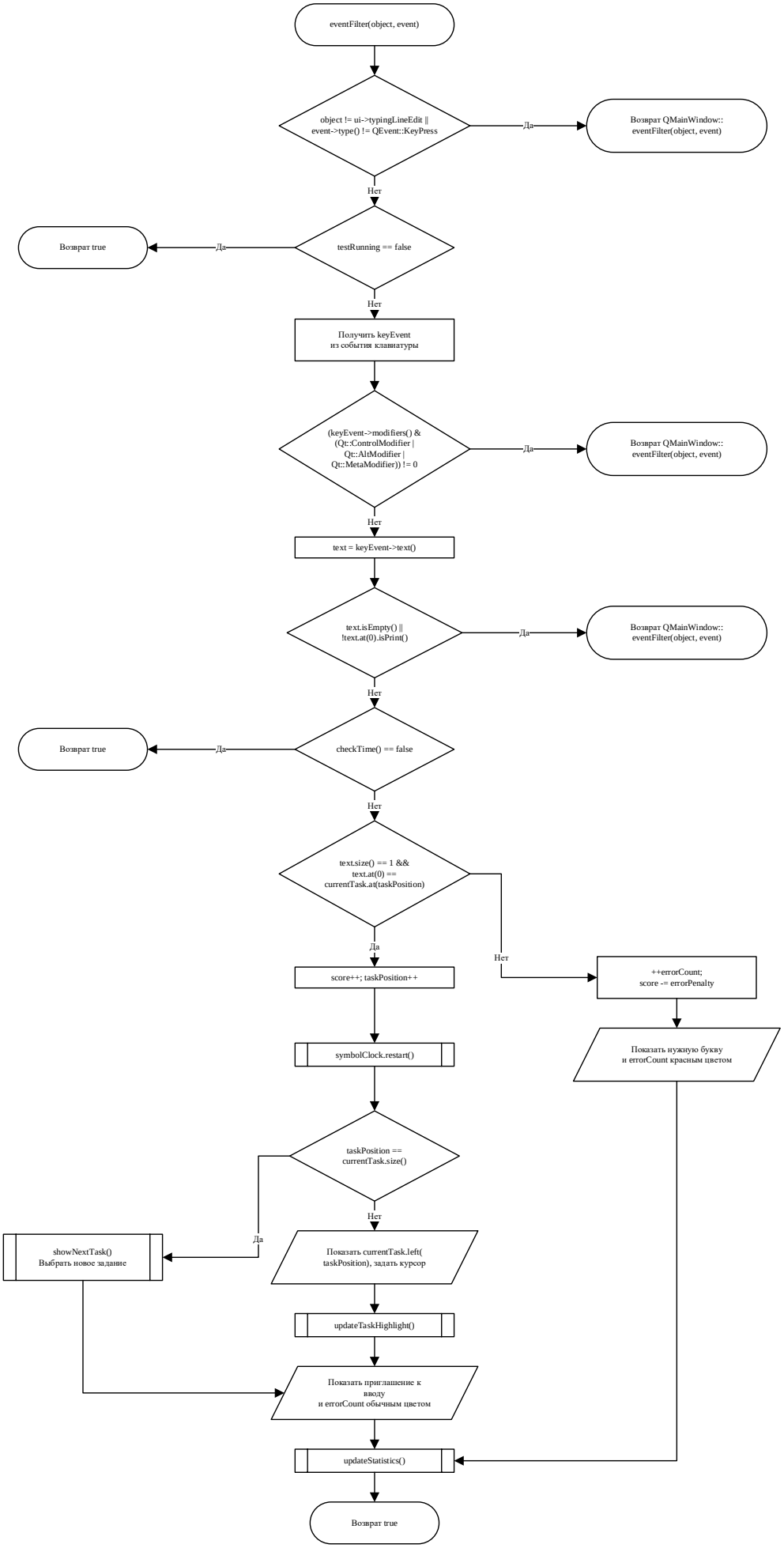


Рисунок Б.2 Обработка ввода

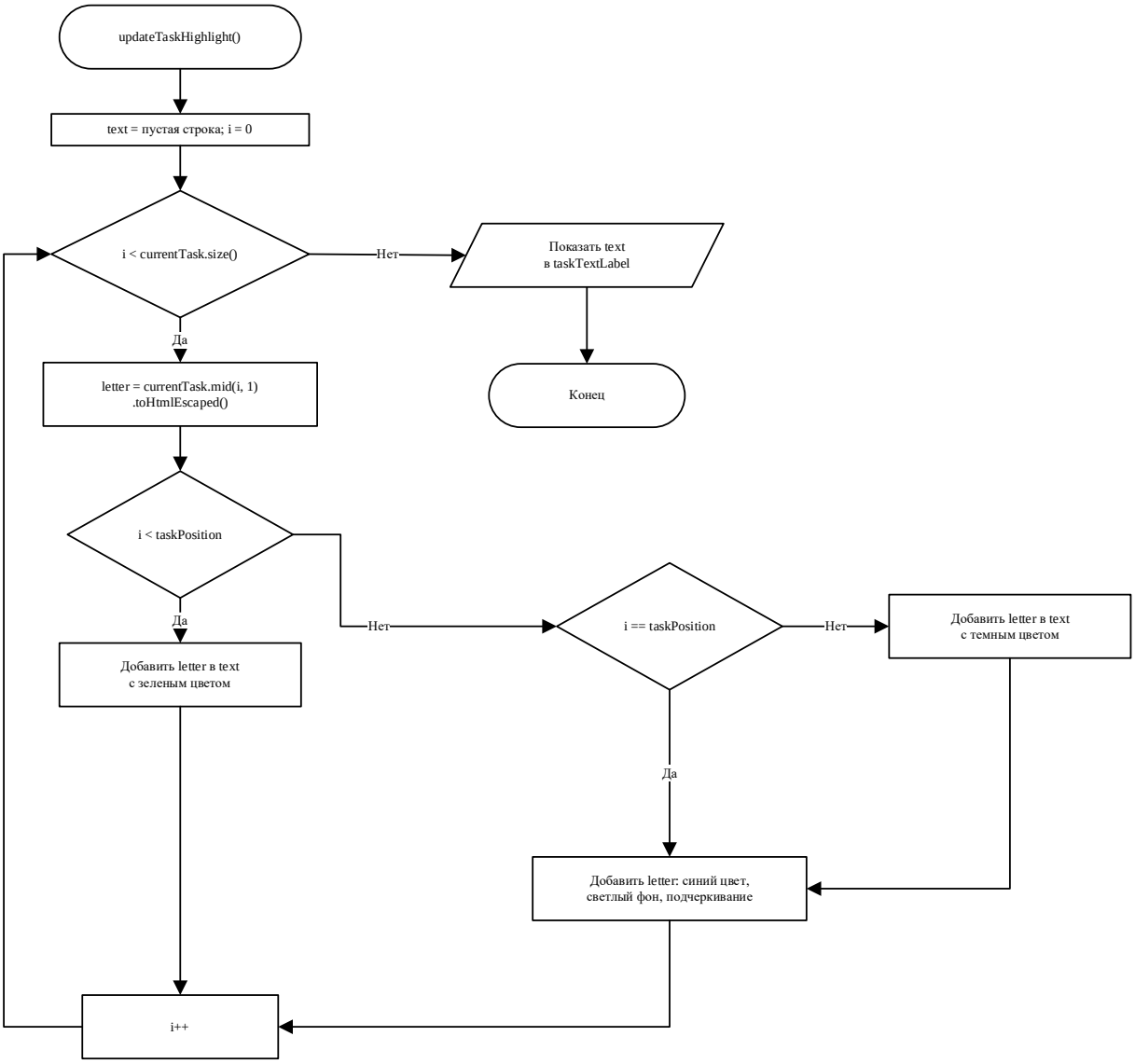


Рисунок Б.3 Подсветка каждого символа в проверяемом слове

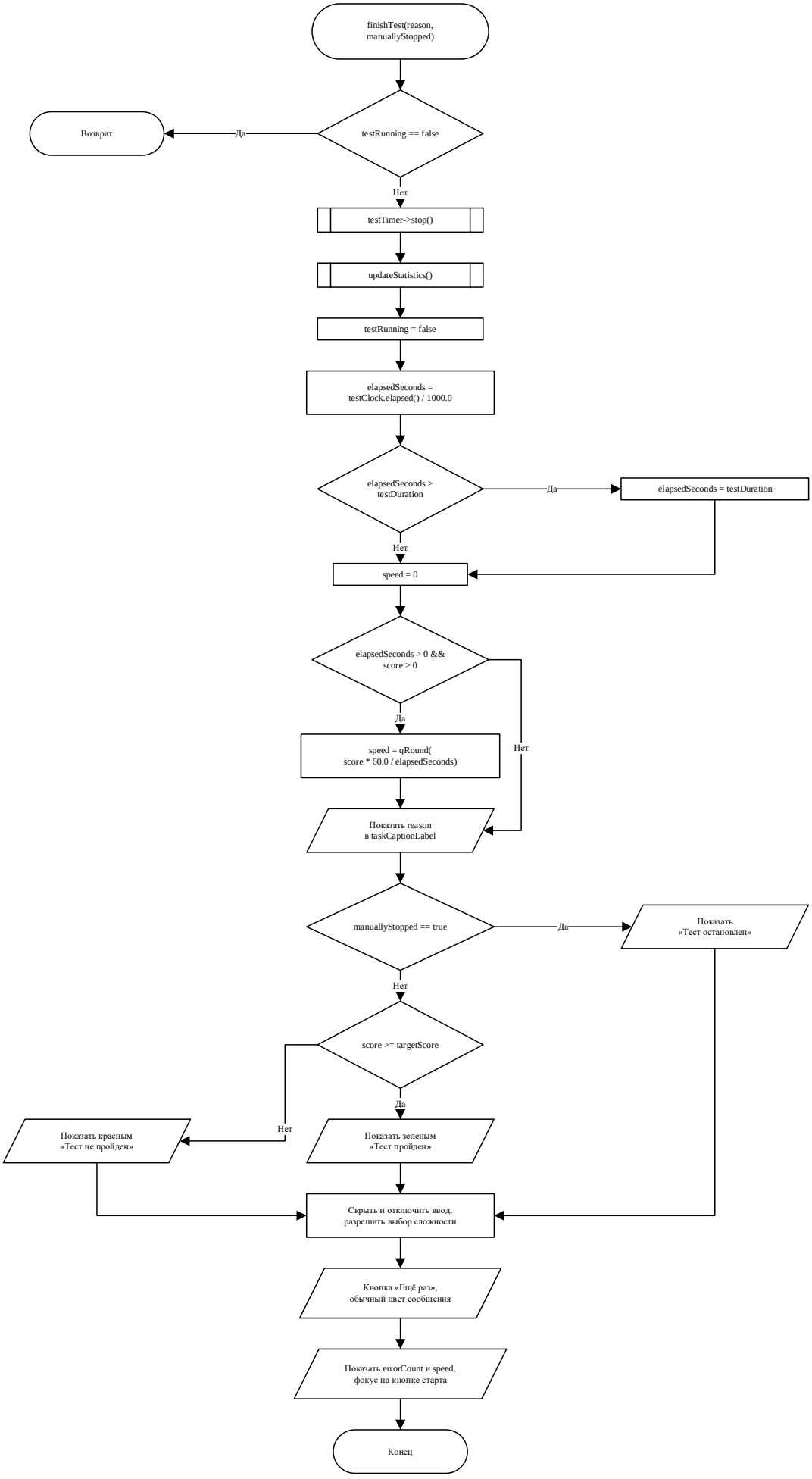


Рисунок Б.4 Завершение и подсчет статистики тренировки

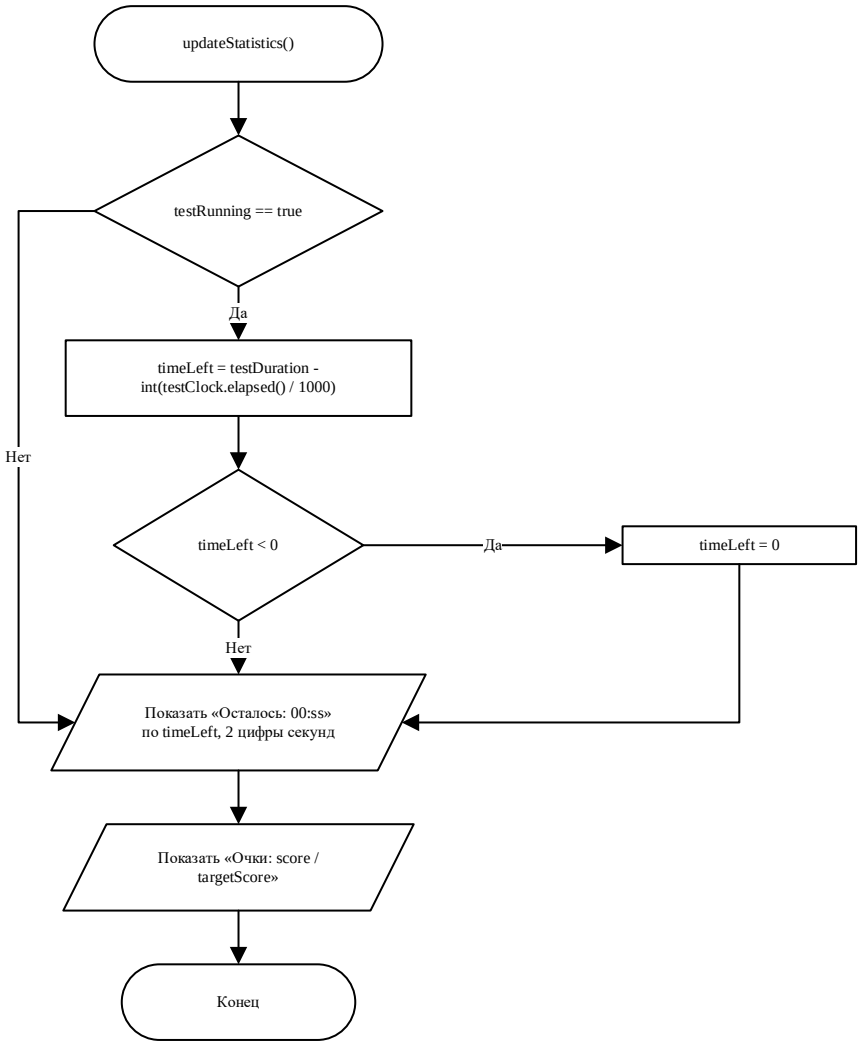


Рисунок Б.5 Обновление статистики тренировки

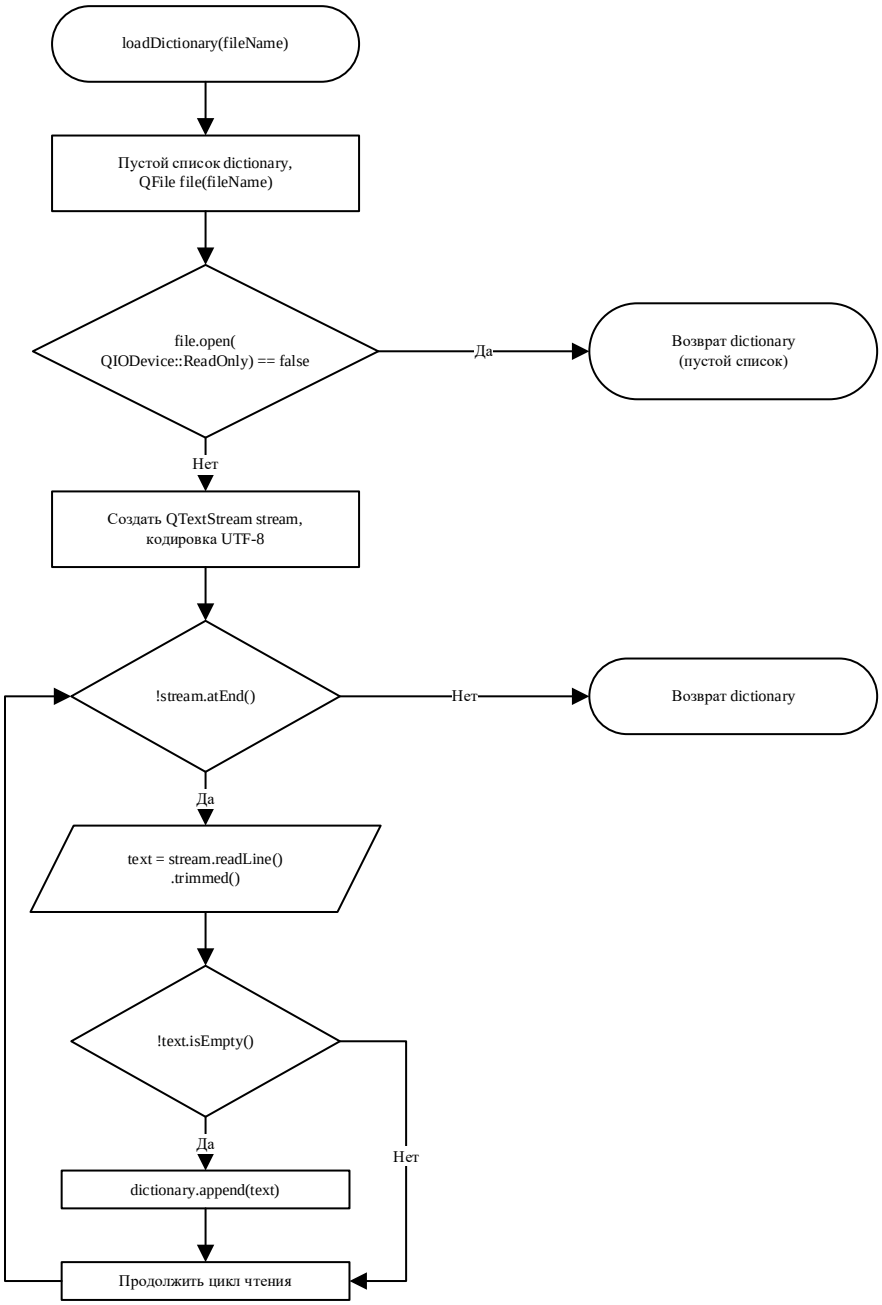


Рисунок Б.6 Загрузка словаря тренажера